Ideation Phase
Empathize & Discover

Date	27.03.2026
Team ID	69bf8031c1eab70902258949
Project Name	Enforcing Mandatory Fields Using UI Policies and Migrating Changes with Updates Sets
Maximum Marks	

Empathy Map: ServiceNow Admin / Developer:

- **THINKS & FEELS:**
- **"I hope I'm in the right Update Set."** (Constant underlying anxiety).
- Desire for a clean UI: "Will making this mandatory annoy the users or help data integrity?"
- Concern about technical debt: "Should I use a UI Policy or a Client Script?" (Hint: UI Policy is almost always better for no-code mandates).
- **Pressure:** Needs the migration to production to be seamless without "skipping" errors.
- **HEARS:**
- **From Users:** "Why can't I save this record? Oh, I missed a field."
- **From Stakeholders:** "We need better data quality; make the 'Assignment Group' mandatory when the state changes to 'In Progress'."
- **From Senior Devs:** "Always check your Update Set logs before migrating."
- **From the System:** The "Success" message after a preview—or the dreaded "Conflict Detected" alert.
- **SEES:**
- **The UI Policy Builder:** Selecting conditions (e.g., `State | is | In Progress`).
- **The Dictionary:** Checking if a field is already mandatory at the table level (and deciding to override it).
- **Update Set Picker:** That tiny dropdown in the header that determines where work is saved.
- **The Progress Bar:** Watching the "Commit Update Set" bar move during migration.
- **SAYS & DOES**
- **Configures UI Policy Actions:** Explicitly sets `Mandatory` to `True` for specific fields.
- **Tests in Sub-Prod:** Mimics a user to ensure the red asterisk appear only when the conditions are met.

- **Names Update Sets clearly:** Uses conventions like `v1.0 - Incident Mandatory Fields - [Date]`.
- **The "Check Twice, Move Once" Routine:** Manually verifying the "Customer Updates" list before closing the set.

Technical Workflow Breakdown:

If you are currently executing these tasks, here is the "Golden Rule" path to ensure your Empathy Map leads to a successful deployment.

1. The Setup (Update Sets)

Before touching the UI Policy, you must capture the work.

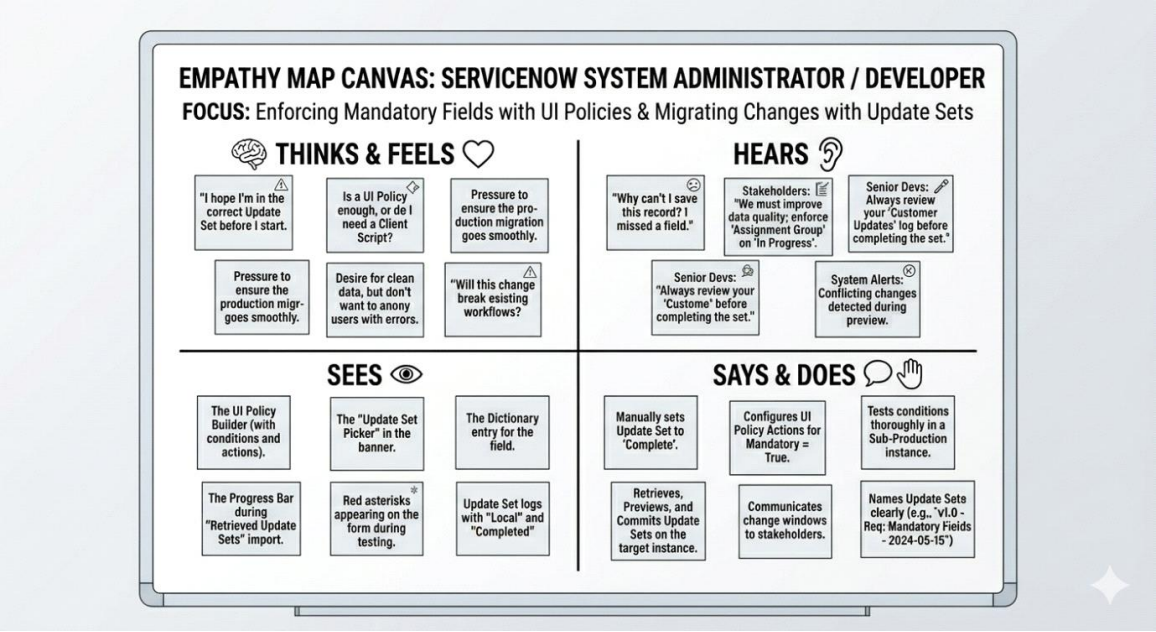
- **Action:** Navigate to **Local Update Sets** > **New**.
- **Tip:** Make it the **Current** set. If you forget this, your UI Policy will end up in "Default," which is a nightmare to extract later.

2. The Logic (UI Policy)

- **Mandatory Fields:** Use the **UI Policy Actions** related list at the bottom of the policy record.
- **Avoid Overlap:** Ensure you aren't fighting a Data Policy or a UI Action. UI Policies are "Client Side" (they happen in the browser), making them great for immediate user feedback.

3. The Migration (Moving the Change)

- **State Change:** Mark the Update Set as **Complete**.
- **The Pull:** Go to the Target Instance (e.g., Test or Prod) and **Retrieve Completed Update Sets**.
- **The Validation:** Click **Preview**. This is where you resolve "collisions" if someone else modified the same form recently.
- **The Final Step: Commit**.


EMPATHY MAP CANVAS: SERVICENOW SYSTEM ADMINISTRATOR / DEVELOPER
FOCUS: Enforcing Mandatory Fields with UI Policies & Migrating Changes with Update Sets
THINKS & FEELS
"I hope I'm in the correct Update Set before I start.
Is a UI Policy enough, or de I need a Client Script?
Pressure to ensure the pro-duction migration goes smoothly.
Pressure to ensure the production migr-goes smoothly.
Desire for clean data, but don't want to anony users with errors.
"Will this change break esisting workflows?
HEARS
"Why can't I save this record? I missed a field."
Stakeholders: "We must improve data quality; enforce 'Assignment Group' on 'In Progress'.
Senior Devs: Always review your 'Customer Updates' log before completing the set."
Senior Devs: "Always review your 'Custome' before completing the set."
System Alerts: Conflicting changes detected during preview.
SEES
The UI Policy Builder (with conditions and actions).
The "Update Set Picker" in the banner.
The Dictionary entry for the field.
The Progress Bar during "Retrieved Update Sets" import.
Red asterisks appearing on the form during testing.
Update Set logs with "Local" and "Completed"
SAYS & DOES
Manually sets Update Set to 'Complete'.
Configures UI Policy Actions for Mandatory = True.
Tests conditions thoroughly in a Sub-Production instance.
Retrieves, Previews, and Commits Update Sets on the target instance.
Communicates change windows to stakeholders.
Names Update Sets clearly (e.g., "v1.0 - Req: Mandatory Fields - 2024-05-15")